

Лабораторная работа №2: Линейная алгебра для Data Science

Дисциплина: Data Science

Студент: Ефимов Сергей Робертович

Подготовка окружения

```
In [28]: import numpy as np
import matplotlib.pyplot as plt

# Общий набор данных (из условия лабораторной)
X = np.array([
    [1, 2], # Аня
    [2, 2], # Борис
    [2, 3], # Вера
    [3, 4], # Глеб
    [4, 5], # Дина
    [5, 7], # Егор
    [6, 8], # Жанна
    [7, 10] # Илья
])
y = np.array([48, 52, 55, 63, 70, 78, 85, 94])

print(f"Размерность X: {X.shape}")
print(f"Размерность y: {y.shape}")
print("Окружение готово!")
```

Размерность X: (8, 2)

Размерность y: (8,)

Окружение готово!

Раздел 1. Векторы: координаты, длина, угол и ортогональность

[Бумара] / [Markdown]

Задание 1.1. Координаты и длина

Даны точки $A(1, 2, 3)$ и $B(3, -4, 6)$. Найдите координаты вектора $\vec{AB}$ и его длину.

[Бумара] / [Markdown]

Правило нахождения координат вектора по координатам его начала и конца:

$$\vec{AB} = B - A.$$

Вычитаем по координатам:

$$\vec{AB} = (3 - 1, -4 - 2, 6 - 3) = (2, -6, 3)$$

Евклидова норма (длина) вектора вычисляется по формуле:

$$\|\vec{AB}\|_2 = \sqrt{2^2 + (-6)^2 + 3^2} = \sqrt{4 + 36 + 9} = \sqrt{49} = 7$$

[Интерпретация]

Направление $A \rightarrow B$ показывает смещение от начальной точки к конечной в трёхмерном пространстве. Длина вектора (норма) не может быть отрицательной, так как она вычисляется как квадратный корень из суммы квадратов координат, а квадрат любого действительного числа неотрицателен, как и результат арифметического квадратного корня. Геометрически это физическое расстояние между точками.

```
In [29]: # Задание 1.1
A = np.array([1, 2, 3])
B = np.array([3, -4, 6])

AB = B - A
length_AB = np.linalg.norm(AB)

print(f"Вектор AB: {AB}")
print(f"Длина вектора AB: {length_AB}")
```

Вектор AB: [2 -6 3]

Длина вектора AB: 7.0

Задание 1.2. Угол между векторами

Найдите угол между векторами $a = (-1, 1, 0)^T$, $b = (1, -2, 2)^T$.

[Бумара] / [Markdown]

Скалярное произведение: $\langle a, b \rangle = (-1)(1) + (1)(-2) + (0)(2) = -1 - 2 + 0 = -3$.

Нормы векторов:

$$\|a\|_2 = \sqrt{(-1)^2 + 1^2 + 0^2} = \sqrt{2}$$

$$\|b\|_2 = \sqrt{1^2 + (-2)^2 + 2^2} = \sqrt{1 + 4 + 4} = \sqrt{9} = 3$$

Косинус угла:

$$\cos \varphi = \frac{\langle a, b \rangle}{\|a\|_2 \|b\|_2} = \frac{-3}{3\sqrt{2}} = -\frac{1}{\sqrt{2}} = -\frac{\sqrt{2}}{2}$$

$$\text{Угол } \varphi = \arccos\left(-\frac{\sqrt{2}}{2}\right) = 135^\circ.$$

[Интерпретация]

Скалярное произведение отрицательное (-3), что однозначно указывает на то, что угол между векторами **тупой** (больше 90° и меньше 180°). Векторы направлены в противоположные стороны относительно начала координат.

```
In [30]: # Задание 1.2
a = np.array([-1, 1, 0])
b = np.array([1, -2, 2])

dot_product = np.dot(a, b)
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)

cos_phi = dot_product / (norm_a * norm_b)
# np.clip защищает от ошибок округления, которые могут дать cos_phi = -1.0000000000
cos_phi_clipped = np.clip(cos_phi, -1.0, 1.0)

angle_rad = np.arccos(cos_phi_clipped)
angle_deg = np.degrees(angle_rad)

print(f"Скалярное произведение: {dot_product}")
print(f"Угол в радианах: {angle_rad:.4f}")
print(f"Угол в градусах: {angle_deg:.1f}")
```

Скалярное произведение: -3
 Угол в радианах: 2.3562
 Угол в градусах: 135.0

Задание 1.3. Ортогональность

При каком значении x векторы $a = (3, 2, -1)^T$, $b = (2, x, 2)^T$ ортогональны?

[Бумага] / [Markdown]

Векторы ортогональны, если их скалярное произведение равно нулю: $\langle a, b \rangle = 0$.

$$3 \cdot 2 + 2 \cdot x + (-1) \cdot 2 = 0$$

$$6 + 2x - 2 = 0$$

$$2x + 4 = 0 \implies x = -2$$

Проверка: $3(2) + 2(-2) + (-1)(2) = 6 - 4 - 2 = 0$.

[Интерпретация]

Нулевое скалярное произведение означает, что проекция одного вектора на другой равна нулю. В контексте данных это говорит об отсутствии линейной согласованности направлений: изменение вдоль одного вектора не даёт никакой информации об

изменении вдоль другого (они независимы в геометрическом смысле).

```
In [31]: # Задание 1.3
a = np.array([3, 2, -1])
x_val = -2
b = np.array([2, x_val, 2])

dot_check = np.dot(a, b)
print(f"Вектор b: {b}")
print(f"Скалярное произведение a и b: {dot_check}")
```

Вектор b: [2 -2 2]

Скалярное произведение a и b: 0

Задание 1.4. Векторы объектов в данных

Запишите векторы признаков Ани и Ильи из общей таблицы. Найдите вектор изменения от Ани к Илье и его норму.

[Бумара] / [Markdown]

Вектор Ани: $v_{\text{Аня}} = (1, 2)$.

Вектор Ильи: $v_{\text{Илья}} = (7, 10)$.

Вектор изменения: $\Delta v = v_{\text{Илья}} - v_{\text{Аня}} = (7 - 1, 10 - 2) = (6, 8)$.

Норма вектора изменения: $\|\Delta v\|_2 = \sqrt{6^2 + 8^2} = \sqrt{36 + 64} = \sqrt{100} = 10$.

[Интерпретация]

Первая координата измеряется в **часах**, вторая — в **штуках** (комплектов упражнений). Смешанная евклидова норма здесь требует осторожной интерпретации, потому что мы складываем квадраты разнородных величин (часы² + штуки²), что физически не вполне корректно. Именно поэтому перед сравнением признаков с разными единицами измерения и масштабами в Data Science выполняют **стандартизацию** (приведение к нулевому среднему и единичной дисперсии), чтобы ни один признак не доминировал в расстоянии только из-за своих числовых значений. К этому мы вернёмся в разделе про PCA.

```
In [32]: # Задание 1.4
v_anya = X[0]
v_ilya = X[-1]

delta_v = v_ilya - v_anya
norm_delta = np.linalg.norm(delta_v)

print(f"Вектор Ани: {v_anya}")
print(f"Вектор Ильи: {v_ilya}")
print(f"Вектор изменения: {delta_v}")
print(f"Норма вектора изменения: {norm_delta}")
```

```
Вектор Ани: [1 2]
Вектор Ильи: [ 7 10]
Вектор изменения: [6 8]
Норма вектора изменения: 10.0
```

Раздел 2. Матрица данных и матричные операции

Задание 2.1. Сложение матриц

Вычислите $A + B$.

[Бумара] / [Markdown]

$$A = \begin{pmatrix} 3 & 5 & 7 \\ 2 & -1 & 0 \\ 4 & 3 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 & 4 \\ 2 & 3 & -2 \\ -1 & 0 & 1 \end{pmatrix}$$

Сложение выполняется поэлементно:

$$A + B = \begin{pmatrix} 3+1 & 5+2 & 7+4 \\ 2+2 & -1+3 & 0+(-2) \\ 4+(-1) & 3+0 & 2+1 \end{pmatrix} = \begin{pmatrix} 4 & 7 & 11 \\ 4 & 2 & -2 \\ 3 & 3 & 3 \end{pmatrix}$$

[Интерпретация]

Складывать две матрицы данных осмысленно, когда они описывают **одни и те же признаки для одних и тех же объектов**, но получены из разных источников.

Например, если одна матрица содержит баллы за первое полугодие, а вторая — за второе, их сумма даст итоговые баллы. Однако одинакового размера **недостаточно**: если в одной матрице столбцы — это «рост» и «вес», а в другой — «доход» и «возраст», то сложение бессмысленно, несмотря на совпадение размерностей $m \times n$.

```
In [33]: # Задание 2.1
A = np.array([[3, 5, 7],
              [2, -1, 0],
              [4, 3, 2]])
```

```

B = np.array([[1, 2, 4],
              [2, 3, -2],
              [-1, 0, 1]])

result = A + B
print("A + B =")
print(result)

```

```

A + B =
[[ 4  7 11]
 [ 4  2 -2]
 [ 3  3  3]]

```

Задание 2.2. Линейная комбинация матриц

Вычислите $2A + 5B$.

[Бумага] / [Markdown]

$$A = \begin{pmatrix} 3 & 5 \\ 4 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 3 \\ 1 & -2 \end{pmatrix}$$

Промежуточный шаг — умножение на скаляры:

$$2A = \begin{pmatrix} 6 & 10 \\ 8 & 2 \end{pmatrix}, \quad 5B = \begin{pmatrix} 10 & 15 \\ 5 & -10 \end{pmatrix}$$

Сложение:

$$2A + 5B = \begin{pmatrix} 6 + 10 & 10 + 15 \\ 8 + 5 & 2 + (-10) \end{pmatrix} = \begin{pmatrix} 16 & 25 \\ 13 & -8 \end{pmatrix}$$

[Интерпретация]

Линейную комбинацию можно рассматривать как **взвешенное объединение двух источников числового сигнала**. Коэффициенты 2 и 5 задают «вклад» каждого источника. Например, если A — оценки эксперта 1, а B — оценки эксперта 2, то $2A + 5B$ соответствует итоговой оценке, в которой мнение второго эксперта учитывается с большим весом.

```

In [34]: # Задание 2.2
A2 = np.array([[3, 5],
               [4, 1]])

B2 = np.array([[2, 3],
               [1, -2]])

result2 = 2 * A2 + 5 * B2
print("2A + 5B =")
print(result2)

```

$2A + 5B =$
[[16 25]
[13 -8]]

Задание 2.3. Произведение матриц и порядок множителей

Найдите AB и BA .

[Бумара] / [Markdown]

$$A = \begin{pmatrix} 1 & 3 & 1 \\ 2 & 0 & 4 \\ 1 & 2 & 3 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 1 & 0 \\ 1 & -1 & 2 \\ 3 & 2 & 1 \end{pmatrix}$$

Элемент $(1, 1)$ произведения AB — скалярное произведение 1-й строки A и 1-го столбца B :

$$(AB)_{11} = 1 \cdot 2 + 3 \cdot 1 + 1 \cdot 3 = 8$$

Полное произведение:

$$AB = \begin{pmatrix} 8 & 0 & 7 \\ 16 & 10 & 4 \\ 13 & 5 & 7 \end{pmatrix}$$

Элемент $(1, 1)$ произведения BA :

$$(BA)_{11} = 2 \cdot 1 + 1 \cdot 2 + 0 \cdot 1 = 4$$

Полное произведение:

$$BA = \begin{pmatrix} 4 & 6 & 6 \\ 1 & 7 & 3 \\ 8 & 11 & 14 \end{pmatrix}$$

Вывод: $AB \neq BA$. Умножение матриц **не коммутативно**.

[Интерпретация]

«Сначала преобразование B , затем A » в общем случае отличается от обратного порядка. Представьте, что B — это поворот изображения, а A — его растяжение. Если сначала повернуть, а потом растянуть по горизонтали, результат будет другим, чем если сначала растянуть, а потом повернуть. Порядок операций имеет значение.

```
In [35]: # Задание 2.3
A3 = np.array([[1, 3, 1],
               [2, 0, 4],
               [1, 2, 3]])

B3 = np.array([[2, 1, 0],
               [1, -1, 2],
```

```

[3, 2, 1]])

AB = A3 @ B3
BA = B3 @ A3

print("AB =")
print(AB)
print("\nBA =")
print(BA)
print(f"\nAB == BA? {np.array_equal(AB, BA)}")

```

```

AB =
[[ 8  0  7]
 [16 10  4]
 [13  5  7]]

```

```

BA =
[[ 4  6  6]
 [ 1  7  3]
 [ 8 11 14]]

```

```

AB == BA? False

```

Задание 2.4. Размерность матрицы данных

Подпишите смысл чисел в размерностях X и y .

[Markdown]

Для $X \in \mathbb{R}^{8 \times 2}$:

- **8** — количество наблюдений (студентов: Аня, Борис, ..., Илья). Каждая строка — один студент.
- **2** — количество признаков (1-й столбец: часы практики, 2-й столбец: комплекты упражнений).

Для $y \in \mathbb{R}^8$:

- **8** элементов соответствуют **строкам** X по порядку: $y_1 = 48$ — балл Ани (1-я строка X), $y_8 = 94$ — балл Ильи (8-я строка X).

[Интерпретация]

Перестановка двух столбцов X меняет смысл коэффициентов будущей модели. Если в исходном порядке β_1 отвечает за часы, а β_2 — за комплекты, то после перестановки столбцов β_1 будет соответствовать комплектам, а β_2 — часам. Числовые значения коэффициентов поменяются местами, хотя само предсказание $\hat{y} = D\beta$ останется прежним. Количество строк при этом не изменяется — объектов по-прежнему восемь.

```

In [36]: # Задание 2.4
print(f"X.shape = {X.shape}")
print(f"y.shape = {y.shape}")

```

```

print(f"\nСредние значения столбцов X:")
print(f"  Часы практики: {X[:, 0].mean():.2f}")
print(f"  Комплекты упражнений: {X[:, 1].mean():.2f}")

XTX = X.T @ X
print(f"\nX^T X =")
print(XTX)
print(f"\nРазмерность X^T X: {XTX.shape}")

```

X.shape = (8, 2)
y.shape = (8,)

Средние значения столбцов X:
Часы практики: 3.75
Комплекты упражнений: 5.12

$X^T X =$
[[144 197]
 [197 271]]

Размерность $X^T X$: (2, 2)

Раздел 3. Линейные отображения

Задание 3.1. Матрица преобразования

Линейное преобразование задано правилом $A(x) = (x_2 - x_3, x_1, x_1 + x_3)^T$. Найдите матрицу этого преобразования в стандартном базисе.

[Бумара] / [Markdown]

Матрица линейного преобразования составляется из образов базисных векторов $e_1 = (1, 0, 0)^T$, $e_2 = (0, 1, 0)^T$, $e_3 = (0, 0, 1)^T$, записанных в столбцы.

Находим образы базисных векторов:

$$A(e_1) = A \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 - 0 \\ 1 \\ 1 + 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}$$

$$A(e_2) = A \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 - 0 \\ 0 \\ 0 + 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

$$A(e_3) = A \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 - 1 \\ 0 \\ 0 + 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix}$$

Составляем матрицу из полученных векторов-столбцов:

$$A = \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

Проверка: Умножим матрицу A на произвольный вектор $x = (x_1, x_2, x_3)^T$:

$$Ax = \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \cdot x_1 + 1 \cdot x_2 + (-1) \cdot x_3 \\ 1 \cdot x_1 + 0 \cdot x_2 + 0 \cdot x_3 \\ 1 \cdot x_1 + 0 \cdot x_2 + 1 \cdot x_3 \end{pmatrix} = \begin{pmatrix} x_2 - x_3 \\ x_1 \\ x_1 + x_3 \end{pmatrix}$$

Результат совпадает с исходным правилом.

[Интерпретация]

Каждый выходной признак является линейной комбинацией входных. Первая строка матрицы $(0, 1, -1)$ показывает, что первый выходной признак получается как $0 \cdot x_1 + 1 \cdot x_2 - 1 \cdot x_3$, то есть разность второго и третьего входных признаков. Вторая строка $(1, 0, 0)$ означает, что второй выходной признак просто копирует первый входной. Третья строка $(1, 0, 1)$ показывает, что третий выходной признак — это сумма первого и третьего входных. В Data Science такие линейные преобразования используются для создания новых признаков (feature engineering) или для перехода в другое пространство представлений.

```
In [37]: # Задание 3.1
A = np.array([[0, 1, -1],
              [1, 0, 0],
              [1, 0, 1]])

# Тестовые векторы
test_vectors = [
    np.array([1, 0, 0]),
    np.array([0, 1, 0]),
    np.array([0, 0, 1]),
    np.array([2, 3, 4])
]

print("Матрица преобразования A:")
print(A)
print("\nПроверка на тестовых векторах:")
for x in test_vectors:
    result = A @ x
    expected = np.array([x[1] - x[2], x[0], x[0] + x[2]])
    match = np.allclose(result, expected)
    print(f"x = {x} → Ax = {result} (ожидается {expected}, совпадает: {match})")
```

Матрица преобразования A :

```
[[ 0  1 -1]
 [ 1  0  0]
 [ 1  0  1]]
```

Проверка на тестовых векторах:

$x = [1 \ 0 \ 0] \rightarrow Ax = [0 \ 1 \ 1]$ (ожидается $[0 \ 1 \ 1]$, совпадает: True)

$x = [0 \ 1 \ 0] \rightarrow Ax = [1 \ 0 \ 0]$ (ожидается $[1 \ 0 \ 0]$, совпадает: True)

$x = [0 \ 0 \ 1] \rightarrow Ax = [-1 \ 0 \ 1]$ (ожидается $[-1 \ 0 \ 1]$, совпадает: True)

$x = [2 \ 3 \ 4] \rightarrow Ax = [-1 \ 2 \ 6]$ (ожидается $[-1 \ 2 \ 6]$, совпадает: True)

Задание 3.2. Образ вектора

Найдите образ вектора $a = (1, -2, 3)^T$ при отображении с матрицей

$$A = \begin{pmatrix} 1 & -1 & 0 \\ 3 & 0 & 1 \\ 1 & 1 & -2 \end{pmatrix}.$$

[Бумара] / [Markdown]

Вычисляем произведение Aa строка за строкой:

$$Aa = \begin{pmatrix} 1 & -1 & 0 \\ 3 & 0 & 1 \\ 1 & 1 & -2 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \\ 3 \end{pmatrix}$$

Первая строка: $1 \cdot 1 + (-1) \cdot (-2) + 0 \cdot 3 = 1 + 2 + 0 = 3$

Вторая строка: $3 \cdot 1 + 0 \cdot (-2) + 1 \cdot 3 = 3 + 0 + 3 = 6$

Третья строка: $1 \cdot 1 + 1 \cdot (-2) + (-2) \cdot 3 = 1 - 2 - 6 = -7$

Итоговый вектор:

$$Aa = \begin{pmatrix} 3 \\ 6 \\ -7 \end{pmatrix}$$

[Интерпретация]

Представим, что $a = (1, -2, 3)^T$ — это три исходных признака объекта (например, возраст, доход, количество покупок). Строки матрицы A — это правила создания трёх новых признаков. Первая строка $(1, -1, 0)$ означает: «новый признак №1 = возраст – доход». Вторая строка $(3, 0, 1)$ означает: «новый признак №2 = 3 × возраст + количество покупок». Третья строка $(1, 1, -2)$ означает: «новый признак №3 = возраст + доход – 2 × количество покупок». Таким образом, матрица A преобразует исходное пространство признаков в новое пространство, где каждый новый признак — это линейная комбинация старых.

```
In [38]: # Задание 3.2
A2 = np.array([[1, -1, 0],
               [3, 0, 1],
               [1, 1, -2]])

a = np.array([1, -2, 3])

result = A2 @ a
print("Матрица A:")
print(A2)
print(f"\nВектор a: {a}")
print(f"Образ вектора Aa: {result}")
```

Матрица A:

```
[[ 1 -1  0]
 [ 3  0  1]
 [ 1  1 -2]]
```

Вектор a: [1 -2 3]

Образ вектора Aa: [3 6 -7]

Раздел 4. Определитель и ранг

Задание 4.1. Определитель

Вычислите определитель матрицы:

$$A = \begin{pmatrix} 5 & 2 & 3 \\ -4 & 3 & 5 \\ 2 & 3 & 4 \end{pmatrix}$$

[Бумара] / [Markdown]

Разложим определитель по первой строке, используя алгебраические дополнения:

$$\det(A) = a_{11} \cdot M_{11} - a_{12} \cdot M_{12} + a_{13} \cdot M_{13}$$

Вычисляем три минора (определители 2×2):

$$M_{11} = \begin{vmatrix} 3 & 5 \\ 3 & 4 \end{vmatrix} = 3 \cdot 4 - 5 \cdot 3 = 12 - 15 = -3$$

$$M_{12} = \begin{vmatrix} -4 & 5 \\ 2 & 4 \end{vmatrix} = (-4) \cdot 4 - 5 \cdot 2 = -16 - 10 = -26$$

$$M_{13} = \begin{vmatrix} -4 & 3 \\ 2 & 3 \end{vmatrix} = (-4) \cdot 3 - 3 \cdot 2 = -12 - 6 = -18$$

Подставляем в формулу разложения:

$$\det(A) = 5 \cdot (-3) - 2 \cdot (-26) + 3 \cdot (-18)$$

$$\det(A) = -15 + 52 - 54 = -17$$

[Интерпретация]

Определитель $\det(A) = -17 \neq 0$, значит матрица **невырожденная** (обратима). Это означает:

1. **Обратная матрица существует** — преобразование можно «откатить» назад.
2. **Трёхмерный объём не схлопывается** — при линейном отображении $y = Ax$ объём любого тела изменяется ровно в $|\det(A)| = 17$ раз, но не обращается в ноль.
3. **Знак «минус»** говорит об изменении ориентации пространства (как если бы мы отразили объект в зеркале).

В контексте данных: если бы матрица A описывала преобразование признаков, то нулевой определитель означал бы, что один из признаков стал линейной комбинацией других — то есть информация потеряна.

```
In [39]: # Задание 4.1
A4 = np.array([[5, 2, 3],
               [-4, 3, 5],
               [2, 3, 4]])

det_A = np.linalg.det(A4)
print(f"Матрица A:\n{A4}")
print(f"\nОпределитель A (через NumPy): {det_A:.6f}")
print(f"Определитель A (округлённый): {round(det_A)}")
print(f"\nМатрица обратима? {abs(det_A) > 1e-10}")
```

Матрица A:

```
[[ 5  2  3]
 [-4  3  5]
 [ 2  3  4]]
```

Определитель A (через NumPy): -17.000000
 Определитель A (округлённый): -17

Матрица обратима? True

Задание 4.2. Ранг двух матриц

Найдите ранги матриц:

$$A_1 = \begin{pmatrix} 1 & 2 & 0 \\ 5 & -1 & 4 \\ 1 & 1 & 1 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 3 & -5 & -1 \\ -2 & 8 & -6 \\ 3 & -7 & 2 \\ 4 & 5 & -3 \end{pmatrix}$$

[Бумара] / [Markdown]

Матрица A_1 (размер 3×3):

Приводим к ступенчатому виду элементарными преобразованиями строк:

$$R_2 \leftarrow R_2 - 5R_1, R_3 \leftarrow R_3 - R_1:$$

$$\begin{pmatrix} 1 & 2 & 0 \\ 0 & -11 & 4 \\ 0 & -1 & 1 \end{pmatrix}$$

$$R_3 \leftarrow R_3 - \frac{1}{11}R_2:$$

$$\begin{pmatrix} 1 & 2 & 0 \\ 0 & -11 & 4 \\ 0 & 0 & \frac{7}{11} \end{pmatrix}$$

Все три строки ненулевые $\rightarrow \text{rank}(A_1) = 3$.

Матрица A_2 (размер 4×3):

$$R_2 \leftarrow R_2 + \frac{2}{3}R_1, R_3 \leftarrow R_3 - R_1, R_4 \leftarrow R_4 - \frac{4}{3}R_1:$$

$$\begin{pmatrix} 3 & -5 & -1 \\ 0 & \frac{14}{3} & -\frac{20}{3} \\ 0 & -2 & 3 \\ 0 & \frac{35}{3} & -\frac{5}{3} \end{pmatrix}$$

$$R_3 \leftarrow R_3 + \frac{3}{7}R_2, R_4 \leftarrow R_4 - \frac{5}{2}R_2:$$

$$\begin{pmatrix} 3 & -5 & -1 \\ 0 & \frac{14}{3} & -\frac{20}{3} \\ 0 & 0 & \frac{1}{7} \\ 0 & 0 & 15 \end{pmatrix}$$

$$R_4 \leftarrow R_4 - 105R_3:$$

$$\begin{pmatrix} 3 & -5 & -1 \\ 0 & \frac{14}{3} & -\frac{20}{3} \\ 0 & 0 & \frac{1}{7} \\ 0 & 0 & 0 \end{pmatrix}$$

Три ненулевые строки $\rightarrow \text{rank}(A_2) = 3$.

[Интерпретация]

- Для A_1 (3×3) максимально возможный ранг — $\min(3, 3) = 3$. Найденный ранг равен максимуму, значит столбцы **линейно независимы**, и матрица невырожденная (что согласуется с $\det(A_1) \neq 0$).

- Для A_2 (4×3) максимально возможный ранг — $\min(4, 3) = 3$. Найденный ранг равен максимуму, значит столбцы **линейно независимы**, хотя строк больше, чем столбцов. Это означает, что среди четырёх строк есть ровно три независимые, а четвёртая — их линейная комбинация.

```
In [40]: # Задание 4.2
A1 = np.array([[1, 2, 0],
               [5, -1, 4],
               [1, 1, 1]])

A2 = np.array([[3, -5, -1],
               [-2, 8, -6],
               [3, -7, 2],
               [4, 5, -3]])

rank_A1 = np.linalg.matrix_rank(A1)
rank_A2 = np.linalg.matrix_rank(A2)

print(f"Матрица A1 (размер {A1.shape}):")
print(A1)
print(f"Ранг A1: {rank_A1}")
print(f"Максимально возможный ранг: {min(A1.shape)}")

print(f"\nМатрица A2 (размер {A2.shape}):")
print(A2)
print(f"Ранг A2: {rank_A2}")
print(f"Максимально возможный ранг: {min(A2.shape)}")
```

Матрица A1 (размер (3, 3)):

```
[[ 1  2  0]
 [ 5 -1  4]
 [ 1  1  1]]
```

Ранг A1: 3

Максимально возможный ранг: 3

Матрица A2 (размер (4, 3)):

```
[[ 3 -5 -1]
 [-2  8 -6]
 [ 3 -7  2]
 [ 4  5 -3]]
```

Ранг A2: 3

Максимально возможный ранг: 3

Раздел 5. Базис и координаты

Задание 5.1. Проверка базиса

Проверьте, образуют ли базис в $\mathbb{R}^3$ векторы $a = (0, 1, 2)^T$, $b = (1, 0, 1)^T$, $c = (-1, 2, 4)^T$.

[Бумара] / [Markdown]

Три вектора в $\mathbb{R}^3$ образуют базис тогда и только тогда, когда они линейно независимы. Это эквивалентно тому, что матрица, составленная из этих векторов-столбцов, имеет ненулевой определитель (или ранг 3).

Составляем матрицу:

$$M = \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & 2 \\ 2 & 1 & 4 \end{pmatrix}$$

Вычисляем определитель разложением по первой строке:

$$\det(M) = 0 \cdot \begin{vmatrix} 0 & 2 \\ 1 & 4 \end{vmatrix} - 1 \cdot \begin{vmatrix} 1 & 2 \\ 2 & 4 \end{vmatrix} + (-1) \cdot \begin{vmatrix} 1 & 0 \\ 2 & 1 \end{vmatrix}$$

$$\det(M) = 0 \cdot (0 \cdot 4 - 2 \cdot 1) - 1 \cdot (1 \cdot 4 - 2 \cdot 2) - 1 \cdot (1 \cdot 1 - 0 \cdot 2)$$

$$\det(M) = 0 - 1 \cdot (4 - 4) - 1 \cdot (1 - 0) = 0 - 0 - 1 = -1$$

Так как $\det(M) = -1 \neq 0$, векторы **образуют базис** в $\mathbb{R}^3$.

[Интерпретация]

Линейная независимость означает, что ни один из трёх векторов нельзя представить как линейную комбинацию двух других. Геометрически это значит, что векторы не лежат в одной плоскости — они «растягивают» всё трёхмерное пространство. Если бы определитель был равен нулю, векторы оказались бы линейно зависимыми (лежали бы в одной плоскости или на одной прямой), и мы потеряли бы одно измерение — не смогли бы однозначно описать произвольную точку в $\mathbb{R}^3$ через координаты в таком «базисе».

```
In [41]: # Задание 5.1
a = np.array([0, 1, 2])
b = np.array([1, 0, 1])
c = np.array([-1, 2, 4])

M = np.column_stack([a, b, c])
det_M = np.linalg.det(M)
rank_M = np.linalg.matrix_rank(M)

print("Матрица M (векторы-столбцы):")
print(M)
print(f"\nОпределитель M: {det_M:.6f}")
print(f"Округлённый определитель: {round(det_M)}")
print(f"Ранг M: {rank_M}")
print(f"\nВекторы образуют базис? {abs(det_M) > 1e-10 and rank_M == 3}")
```

Матрица M (векторы-столбцы):

```
[[ 0  1 -1]
 [ 1  0  2]
 [ 2  1  4]]
```

Определитель M : -1.000000

Округлённый определитель: -1

Ранг M : 3

Векторы образуют базис? True

Задание 5.2. Координаты в новом базисе

Разложите $x = (1, -4, 4)^T$ по векторам $p = (2, 1, -1)^T$, $r = (1, -1, 1)^T$, $q = (0, 3, 2)^T$.

[Бумара] / [Markdown]

Ищем коэффициенты α, β, γ такие, что:

$$x = \alpha p + \beta r + \gamma q$$

Это равносильно решению системы линейных уравнений $M \cdot [\alpha, \beta, \gamma]^T = x$, где:

$$M = \begin{pmatrix} 2 & 1 & 0 \\ 1 & -1 & 3 \\ -1 & 1 & 2 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ -4 \\ 4 \end{pmatrix}$$

Сначала проверим, что определитель матрицы M ненулевой:

$$\det(M) = 2 \cdot ((-1) \cdot 2 - 3 \cdot 1) - 1 \cdot (1 \cdot 2 - 3 \cdot (-1)) + 0$$

$$\det(M) = 2 \cdot (-2 - 3) - 1 \cdot (2 + 3) = 2 \cdot (-5) - 5 = -10 - 5 = -15$$

Так как $\det(M) = -15 \neq 0$, система имеет единственное решение. Решаем методом Крамера:

$$\alpha = \frac{\det(M_\alpha)}{\det(M)}, \quad \beta = \frac{\det(M_\beta)}{\det(M)}, \quad \gamma = \frac{\det(M_\gamma)}{\det(M)}$$

где $M_\alpha, M_\beta, M_\gamma$ — матрицы, полученные заменой соответствующего столбца M на вектор x .

$$M_\alpha = \begin{pmatrix} 1 & 1 & 0 \\ -4 & -1 & 3 \\ 4 & 1 & 2 \end{pmatrix}$$

$$\det(M_\alpha) = 1 \cdot (-2 - 3) - 1 \cdot (-8 - 12) + 0 = -5 + 20 = 15$$

$$\alpha = \frac{15}{-15} = -1$$

$$M_\beta = \begin{pmatrix} 2 & 1 & 0 \\ 1 & -4 & 3 \\ -1 & 4 & 2 \end{pmatrix}$$

$$\det(M_\beta) = 2 \cdot (-8 - 12) - 1 \cdot (2 + 3) + 0 = -40 - 5 = -45$$

$$\beta = \frac{-45}{-15} = 3$$

$$M_\gamma = \begin{pmatrix} 2 & 1 & 1 \\ 1 & -1 & -4 \\ -1 & 1 & 4 \end{pmatrix}$$

$$\det(M_\gamma) = 2 \cdot (-4 + 4) - 1 \cdot (4 - 4) + 1 \cdot (1 - 1) = 0 - 0 + 0 = 0$$

$$\gamma = \frac{0}{-15} = 0$$

Ответ: $\alpha = -1, \beta = 3, \gamma = 0$.

Проверка:

$$x = -1 \cdot p + 3 \cdot r + 0 \cdot q = -1 \cdot \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} + 3 \cdot \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix} + 0 \cdot \begin{pmatrix} 0 \\ 3 \\ 2 \end{pmatrix} = \begin{pmatrix} -2 \\ -1 \\ 1 \end{pmatrix} + \begin{pmatrix} 3 \\ -3 \\ 3 \end{pmatrix} =$$

[Интерпретация]

Сам геометрический вектор x — это абстрактный объект в пространстве, не зависящий от выбора системы координат. Однако его **координаты** (числа α, β, γ) полностью определяются выбранным базисом. В стандартном базисе e_1, e_2, e_3 вектор x имеет координаты $(1, -4, 4)$. В базисе p, r, q тот же самый вектор имеет координаты $(-1, 3, 0)$. Это как если бы мы описывали местоположение объекта: в одной системе координат (например, «от входа») он находится в точке $(1, -4, 4)$, а в другой системе («от угла здания») — в точке $(-1, 3, 0)$. Объект один и тот же, но числа разные.

```
In [42]: # Задание 5.2
p = np.array([2, 1, -1])
r = np.array([1, -1, 1])
q = np.array([0, 3, 2])
x = np.array([1, -4, 4])

M2 = np.column_stack([p, r, q])

# Решаем систему M2 @ [alpha, beta, gamma] = x
coords = np.linalg.solve(M2, x)
alpha, beta, gamma = coords

print("Матрица M (векторы-столбцы p, r, q):")
print(M2)
print(f"\nВектор x: {x}")
```

```

print(f"\nКоординаты в базисе (p, r, q):")
print(f"  α = {alpha:.6f}")
print(f"  β = {beta:.6f}")
print(f"  γ = {gamma:.6f}")

# Проверка
x_reconstructed = alpha * p + beta * r + gamma * q
print(f"\nПроверка: α·p + β·r + γ·q = {x_reconstructed}")
print(f"Совпадает с x? {np.allclose(x_reconstructed, x)}")

```

Матрица M (векторы-столбцы p, r, q):

```

[[ 2  1  0]
 [ 1 -1  3]
 [-1  1  2]]

```

Вектор x: [1 -4 4]

Координаты в базисе (p, r, q):

```

α = -1.000000
β = 3.000000
γ = 0.000000

```

Проверка: $\alpha \cdot p + \beta \cdot r + \gamma \cdot q = [1. \ -4. \ 4.]$

Совпадает с x? True

Раздел 6. Системы линейных уравнений

Задание 6.1. Квадратная система

Решите методом Гаусса систему:

$$\begin{cases} 2x - y + z = 2 \\ 3x + 2y + 2z = -2 \\ x - 2y + z = 1 \end{cases}$$

[Бумара] / [Markdown]

Запишем расширенную матрицу системы:

$$\left(\begin{array}{ccc|c} 2 & -1 & 1 & 2 \\ 3 & 2 & 2 & -2 \\ 1 & -2 & 1 & 1 \end{array} \right)$$

1. Поменяем местами 1-ю и 3-ю строки ($R_1 \leftrightarrow R_3$), чтобы получить 1 на первой позиции:

$$\left(\begin{array}{ccc|c} 1 & -2 & 1 & 1 \\ 3 & 2 & 2 & -2 \\ 2 & -1 & 1 & 2 \end{array} \right)$$

2. Обнулим элементы под главной диагональю в первом столбце: $R_2 \leftarrow R_2 - 3R_1$
 $R_3 \leftarrow R_3 - 2R_1$

$$\left(\begin{array}{ccc|c} 1 & -2 & 1 & 1 \\ 0 & 8 & -1 & -5 \\ 0 & 3 & -1 & 0 \end{array} \right)$$

3. Обнулим элемент под главной диагональю во втором столбце: $R_3 \leftarrow R_3 - \frac{3}{8}R_2$

$$\left(\begin{array}{ccc|c} 1 & -2 & 1 & 1 \\ 0 & 8 & -1 & -5 \\ 0 & 0 & -\frac{5}{8} & \frac{15}{8} \end{array} \right)$$

Обратный ход: Из 3-й строки: $-\frac{5}{8}z = \frac{15}{8} \implies z = -3$. Из 2-й строки:

$$8y - z = -5 \implies 8y - (-3) = -5 \implies 8y = -8 \implies y = -1. \text{ Из 1-й строки:}$$

$$x - 2y + z = 1 \implies x - 2(-1) + (-3) = 1 \implies x + 2 - 3 = 1 \implies x = 2.$$

Проверка подстановкой:

1. $2(2) - (-1) + (-3) = 4 + 1 - 3 = 2$ (верно)
2. $3(2) + 2(-1) + 2(-3) = 6 - 2 - 6 = -2$ (верно)
3. $1(2) - 2(-1) + (-3) = 2 + 2 - 3 = 1$ (верно)

[Интерпретация]

Нулевой вектор невязки ($Ax - b = 0$) означает, что найденный вектор x **точно** удовлетворяет всем уравнениям системы одновременно. Геометрически в трёхмерном пространстве каждое уравнение задаёт плоскость. Нулевая невязка подтверждает, что все три плоскости пересекаются ровно в одной общей точке $(2, -1, -3)$.

```
In [43]: # Задание 6.1
A1 = np.array([[2, -1, 1],
               [3, 2, 2],
               [1, -2, 1]])

b1 = np.array([2, -2, 1])

# Решение системы
x1 = np.linalg.solve(A1, b1)
print(f"Решение системы x: {x1}")

# Вычисление невязки
residual1 = A1 @ x1 - b1
print(f"Невязка (A @ x - b): {residual1}")
print(f"Норма невязки: {np.linalg.norm(residual1):.2e}")
```

```
Решение системы x: [ 2. -1. -3.]
Невязка (A @ x - b): [0. 0. 0.]
Норма невязки: 0.00e+00
```

Задание 6.2. Недоопределённая система

Решите систему:

$$\begin{cases} 3x_1 - 4x_2 + 6x_3 = -7 \\ 2x_1 + 5x_2 + 4x_3 = 12 \end{cases}$$

[Бумара] / [Markdown]

Положим $x_3 = t$ (свободная переменная). Перенесём слагаемые с t в правую часть:

$$\begin{cases} 3x_1 - 4x_2 = -7 - 6t \\ 2x_1 + 5x_2 = 12 - 4t \end{cases}$$

Умножим первое уравнение на 2, а второе на 3, чтобы выровнять коэффициенты при x_1 :

$$\begin{cases} 6x_1 - 8x_2 = -14 - 12t \\ 6x_1 + 15x_2 = 36 - 12t \end{cases}$$

Вычтем первое уравнение из второго:

$$(6x_1 + 15x_2) - (6x_1 - 8x_2) = (36 - 12t) - (-14 - 12t)$$

$$23x_2 = 50 \implies x_2 = \frac{50}{23}$$

Подставим x_2 во второе исходное уравнение, чтобы найти x_1 :

$$2x_1 + 5\left(\frac{50}{23}\right) = 12 - 4t$$

$$2x_1 + \frac{250}{23} = \frac{276}{23} - 4t$$

$$2x_1 = \frac{26}{23} - 4t \implies x_1 = \frac{13}{23} - 2t$$

Общее семейство решений:

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} \frac{13}{23} - 2t \\ \frac{50}{23} \\ t \end{pmatrix}, \quad t \in \mathbb{R}$$

[Интерпретация]

Два независимых линейных уравнения в трёхмерном пространстве описывают две плоскости. Поскольку плоскости не параллельны, они пересекаются по **прямой линии**, а не в одной точке. Параметр t позволяет нам «скользить» вдоль этой прямой, генерируя бесконечное множество допустимых решений. Одно уравнение «съедает»

одну степень свободы, поэтому из трёх переменных одна остаётся свободной.

```
In [44]: # Задание 6.2
A2 = np.array([[3, -4, 6],
               [2, 5, 4]])

b2 = np.array([-7, 12])

def get_solution_6_2(t):
    x3 = t
    x2 = 50 / 23
    x1 = 13 / 23 - 2 * t
    return np.array([x1, x2, x3])

print("Проверка параметрического решения для разных значений t:")
for t in [0.0, 1.0, -2.5]:
    x = get_solution_6_2(t)
    residual = A2 @ x - b2
    print(f"t = {t:5.1f} | x = [{x[0]:6.3f}, {x[1]:6.3f}, {x[2]:6.3f}] | Невязка =
```

Проверка параметрического решения для разных значений t:

t = 0.0 | x = [0.565, 2.174, 0.000] | Невязка = [0. 0.]

t = 1.0 | x = [-1.435, 2.174, 1.000] | Невязка = [0. 0.]

t = -2.5 | x = [5.565, 2.174, -2.500] | Невязка = [0. 0.]

Раздел 7. Проекция

Задание 7.1. Проекция на прямую

Пусть $y = (3, 1)^T$, $a = (1, 1)^T$.

[Бумага] / [Markdown]

Найдём коэффициент проекции c по формуле $c = \frac{a^T y}{a^T a}$:

$$a^T y = 1 \cdot 3 + 1 \cdot 1 = 4$$

$$a^T a = 1 \cdot 1 + 1 \cdot 1 = 2$$

$$c = \frac{4}{2} = 2$$

Тогда проекция $\hat{y}$ равна:

$$\hat{y} = c \cdot a = 2 \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

Найдём вектор остатка (ошибки) e :

$$e = y - \hat{y} = \begin{pmatrix} 3 \\ 1 \end{pmatrix} - \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Проверим ортогональность остатка направляющему вектору прямой a :

$$a^T e = 1 \cdot 1 + 1 \cdot (-1) = 1 - 1 = 0$$

Условие ортогональности выполнено.

[Интерпретация]

Проекция $\hat{y}$ является **ближайшей** к вектору y точкой на прямой, натянутой на вектор a . Это следует из теоремы Пифагора: любой другой вектор на этой прямой можно представить как $\hat{y} + \delta a$. Расстояние до y будет равно $\|y - (\hat{y} + \delta a)\|^2 = \|e - \delta a\|^2 = \|e\|^2 + \|\delta a\|^2$ (поскольку e и a ортогональны). Это значение минимально только при $\delta = 0$. В Data Science этот принцип лежит в основе метода наименьших квадратов: мы ищем такие параметры модели, чтобы вектор предсказаний был ортогональной проекцией вектора истинных значений на пространство, натянутое на признаки.

```
In [45]: # Задание 7.1
import matplotlib.pyplot as plt

y = np.array([3, 1])
a = np.array([1, 1])

# Вычисления
c = np.dot(a, y) / np.dot(a, a)
y_hat = c * a
e = y - y_hat

print(f"Коэффициент c: {c}")
print(f"Проекция y_hat: {y_hat}")
print(f"Остаток e: {e}")
print(f"Скалярное произведение a^T e: {np.dot(a, e)}")

# Визуализация
plt.figure(figsize=(6, 6))

# Рисуем векторы от начала координат
plt.quiver(0, 0, a[0], a[1], angles='xy', scale_units='xy', scale=1, color='blue',
plt.quiver(0, 0, y[0], y[1], angles='xy', scale_units='xy', scale=1, color='red',
plt.quiver(0, 0, y_hat[0], y_hat[1], angles='xy', scale_units='xy', scale=1, color=

# Рисуем вектор остатка, начиная от конца проекции
plt.quiver(y_hat[0], y_hat[1], e[0], e[1], angles='xy', scale_units='xy', scale=1,

# Продлеваем прямую 'a' для наглядности
t = np.linspace(-1, 3, 100)
plt.plot(t * a[0], t * a[1], 'b--', alpha=0.5)

# Настройка осей и сетки
plt.xlim(-1, 4)
plt.ylim(-1, 4)
plt.grid(True, linestyle=':', alpha=0.7)
plt.axhline(0, color='black', linewidth=0.8)
```

```
plt.axvline(0, color='black', linewidth=0.8)
plt.legend(loc='upper left')
plt.title("Геометрическая интерпретация проекции")
plt.show()
```

Коэффициент c : 2.0

Проекция y_hat : [2. 2.]

Остаток e : [1. -1.]

Скалярное произведение $a^T e$: 0.0



Раздел 8. Метод наименьших квадратов

Задание 8.1. Матричная запись регрессии

[Бумара] / [Markdown]

Добавим к матрице признаков X столбец единиц для свободного члена β_0 :

$$D = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 4 \\ 1 & 4 & 5 \\ 1 & 5 & 7 \\ 1 & 6 & 8 \\ 1 & 7 & 10 \end{pmatrix} \in \mathbb{R}^{8 \times 3}, \quad \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix}$$

Предсказания имеют вид $\hat{y} = D\beta$. Выпишем первую и последнюю строки в скалярной форме:

Первая строка (Аня):

$$\hat{y}_1 = \beta_0 \cdot 1 + \beta_1 \cdot 1 + \beta_2 \cdot 2 = \beta_0 + \beta_1 + 2\beta_2$$

Последняя строка (Илья):

$$\hat{y}_8 = \beta_0 \cdot 1 + \beta_1 \cdot 7 + \beta_2 \cdot 10 = \beta_0 + 7\beta_1 + 10\beta_2$$

Роли коэффициентов:

- β_0 — **свободный член (интерцепт)**: базовый уровень итогового балла, когда и часы практики, и комплекты упражнений равны нулю. Измеряется в **баллах**.
- β_1 — **коэффициент при часах практики**: на сколько баллов в среднем увеличивается итоговый балл при увеличении числа часов практики на 1 (при фиксированном числе комплектов). Измеряется в **баллах/час**.
- β_2 — **коэффициент при комплектах упражнений**: на сколько баллов в среднем увеличивается итоговый балл при увеличении числа комплектов на 1 (при фиксированном числе часов). Измеряется в **баллах/комплект**.

[Интерпретация]

Коэффициенты β_1 и β_2 описывают **статистическую связь** в нашем небольшом наборе из 8 наблюдений, а не причинно-следственный эффект. Мы не можем утверждать, что «добавление часа практики вызывает рост балла на β_1 » — возможно, более мотивированные студенты и больше практикуются, и лучше сдают. Коэффициенты регрессии — это компактное описание закономерности в имеющихся данных.

```
In [46]: # Задание 8.1
# Явно определяем данные, чтобы избежать проблем с размерностями
X = np.array([
    [1, 2], # Аня
    [2, 2], # Борис
    [2, 3], # Вера
    [3, 4], # Глеб
    [4, 5], # Дина
    [5, 7], # Егор
```

```

    [6, 8],    # Жанна
    [7, 10]   # Илья
])

y = np.array([48, 52, 55, 63, 70, 78, 85, 94])

print(f"Размер X: {X.shape}")
print(f"Размер y: {y.shape}")

# Добавляем столбец единиц для свободного члена
D = np.column_stack([np.ones(X.shape[0]), X])

print(f"\nМатрица D (размер): {D.shape}")
print(D)

# Решаем задачу наименьших квадратов
beta, residuals, rank, s = np.linalg.lstsq(D, y, rcond=None)
beta_0, beta_1, beta_2 = beta

print(f"\nКоэффициенты модели:")
print(f"   $\beta_0$  (интерцепт) = {beta_0:.4f}")
print(f"   $\beta_1$  (часы)      = {beta_1:.4f}")
print(f"   $\beta_2$  (комплекты) = {beta_2:.4f}")

# Вычисляем предсказания
y_hat = D @ beta
print(f"\nПредсказания  $\hat{y}$ : {y_hat}")
print(f"Истинные значения y: {y}")

```

Размер X: (8, 2)

Размер y: (8,)

Матрица D (размер): (8, 3)

```

[[ 1.  1.  2.]
 [ 1.  2.  2.]
 [ 1.  2.  3.]
 [ 1.  3.  4.]
 [ 1.  4.  5.]
 [ 1.  5.  7.]
 [ 1.  6.  8.]
 [ 1.  7. 10.]]

```

Коэффициенты модели:

β_0 (интерцепт) = 38.9787

β_1 (часы) = 4.8936

β_2 (комплекты) = 2.1064

Предсказания $\hat{y}$: [48.08510638 52.9787234 55.08510638 62.08510638 69.08510638 78.19148936

85.19148936 94.29787234]

Истинные значения y: [48 52 55 63 70 78 85 94]

Задание 8.2. Невязка и качество приближения

[Бумага]

Возьмём для примера **Веру** (3-й студент): $x = (2, 3)$, истинный балл $y_3 = 55$.

Предсказание:

$$\hat{y}_3 = \beta_0 + \beta_1 \cdot 2 + \beta_2 \cdot 3$$

Остаток:

$$e_3 = y_3 - \hat{y}_3$$

(Конкретные числа подставим после вычисления коэффициентов в Python)

[Markdown]

Определим метрики качества приближения:

SSE (Sum of Squared Errors) — сумма квадратов ошибок:

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

RMSE (Root Mean Squared Error) — среднеквадратичная ошибка:

$$RMSE = \sqrt{\frac{SSE}{n}}$$

R^2 (коэффициент детерминации) — доля дисперсии, объяснённая моделью:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{SSE}{SST}$$

где SST — общая сумма квадратов отклонений от среднего.

[Интерпретация]

Высокое значение R^2 (близкое к 1) означает, что модель хорошо описывает **именно эти 8 наблюдений**. Однако это **не гарантирует** хорошего качества на новых данных: на маленькой выборке модель может «переобучиться» под шум. Для оценки обобщающей способности нужны либо отдельные тестовые данные, либо кросс-валидация.

```
In [47]: # Задание 8.2
# Вычисляем остатки (используем y_hat из предыдущей ячейки)
residuals_8_2 = y - y_hat

# Метрики качества
SSE = np.sum(residuals_8_2 ** 2)
RMSE = np.sqrt(SSE / len(y))
y_mean = np.mean(y)
SST = np.sum((y - y_mean) ** 2)
R2 = 1 - SSE / SST
```

```

print(f"Пример для Веры (3-й студент):")
print(f"  Истинный балл: {y[2]}")
print(f"  Предсказанный: {y_hat[2]:.4f}")
print(f"  Остаток: {residuals_8_2[2]:.4f}")

print(f"\nМетрики качества модели:")
print(f"  SSE = {SSE:.4f}")
print(f"  RMSE = {RMSE:.4f}")
print(f"  R² = {R2:.4f}")

# График "наблюдение – предсказание"
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Левый: наблюдение vs предсказание
axes[0].scatter(y, y_hat, color='steelblue', s=80, zorder=3)
axes[0].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', label='Идеальное совпад')
axes[0].set_xlabel('Истинный балл y')
axes[0].set_ylabel('Предсказанный балл ŷ')
axes[0].set_title('Наблюдение vs Предсказание')
axes[0].legend()
axes[0].grid(True, linestyle=':', alpha=0.7)

# Правый: график остатков
student_names = ['Аня', 'Борис', 'Вера', 'Глеб', 'Дина', 'Егор', 'Жанна', 'Илья']
axes[1].bar(student_names, residuals_8_2, color='steelblue')
axes[1].axhline(0, color='red', linestyle='--')
axes[1].set_xlabel('Студент')
axes[1].set_ylabel('Остаток (y - ŷ)')
axes[1].set_title('Остатки по студентам')
axes[1].grid(True, linestyle=':', alpha=0.7, axis='y')

plt.tight_layout()
plt.show()

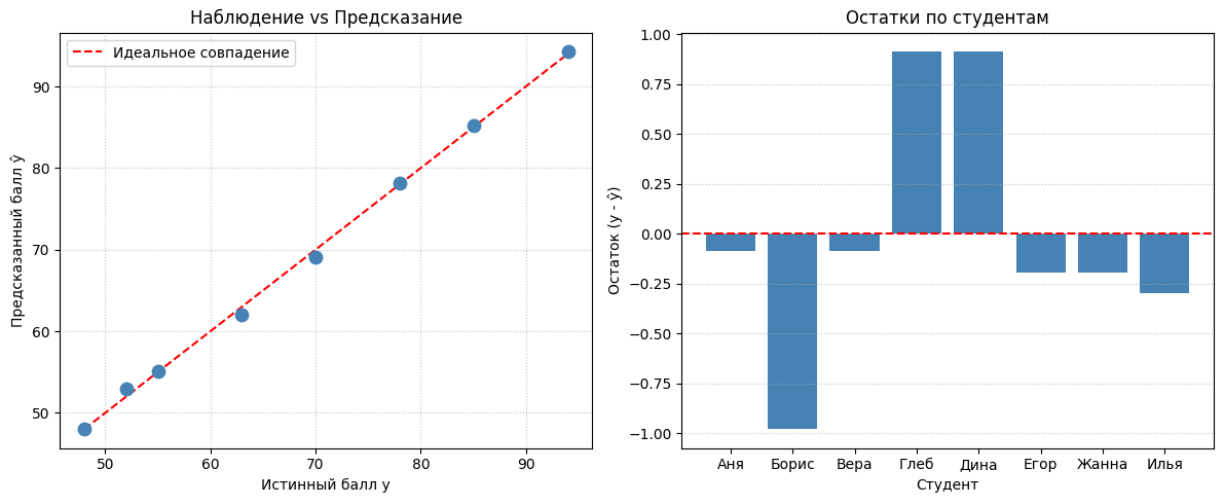
```

Пример для Веры (3-й студент):

Истинный балл: 55
 Предсказанный: 55.0851
 Остаток: -0.0851

Метрики качества модели:

SSE = 2.8085
 RMSE = 0.5925
 R² = 0.9985



Задание 8.3. Регрессия как проекция

[Python]

Проверим два условия:

1. $D^T(y - \hat{y}) \approx 0$ — остаток ортогонален пространству столбцов D .
2. $\hat{y} \in \mathcal{R}(D)$ — предсказания лежат в пространстве столбцов D .

[Интерпретация]

«Вектор предсказаний $\hat{y}$ является **проекцией** вектора результатов y на пространство столбцов матрицы D , потому что... остаток $e = y - \hat{y}$ ортогонален всем столбцам D (условие $D^T e = 0$), а значит $\hat{y}$ — это ближайшая к y точка в пространстве $\mathcal{R}(D)$ в смысле евклидовой нормы. Именно это и означает геометрическая проекция».

```
In [48]: # Задание 8.3
# Проверка ортогональности остатка столбцам D
orthogonality_check = D.T @ residuals_8_2
print("Проверка условия  $D^T (y - \hat{y}) \approx 0$ :")
print(f" Вектор: {orthogonality_check}")
print(f" Норма: {np.linalg.norm(orthogonality_check):.2e}")
print(f" Условие выполнено? {np.linalg.norm(orthogonality_check) < 1e-8}")

# Проверка принадлежности  $\hat{y}$  пространству столбцов D
#  $\hat{y} \in \mathcal{R}(D) \iff$  существует в такой, что  $\hat{y} = D\beta$ 
# Это выполняется по построению:  $\hat{y} = D @ \beta$ 
y_hat_reconstructed = D @ beta
print(f"\nПроверка  $\hat{y} \in \mathcal{R}(D)$ :")
print(f"  $\hat{y}$  совпадает с  $D\beta$ ? {np.allclose(y_hat, y_hat_reconstructed)})")

print("\n" + "="*60)
print("ИТОГ РАЗДЕЛА 8:")
print(f"Модель:  $y \approx \{beta\_0\} + \{beta\_1\} \cdot (\text{часы}) + \{beta\_2\} \cdot (\text{комплекты})$ ")
print(f"Качество:  $R^2 = \{R2\}$ , RMSE =  $\{RMSE\}$  баллов")
print("="*60)
```

Проверка условия $D^T (y - \hat{y}) \approx 0$:

Вектор: [2.20268248e-13 8.02913291e-13 1.09423581e-12]

Норма: 1.37e-12

Условие выполнено? True

Проверка $\hat{y} \in R(D)$:

$\hat{y}$ совпадает с $D\beta$? True

=====

ИТОГ РАЗДЕЛА 8:

Модель: $y \approx 38.98 + 4.89 \cdot (\text{часы}) + 2.11 \cdot (\text{комплекты})$

Качество: $R^2 = 0.9985$, RMSE = 0.59 баллов

=====

Раздел 9. Базовое PCA

Зачем это нужно

Часы практики и число выполненных комплектов в нашем наборе тесно связаны. PCA ищет ортонормированные направления, вдоль которых стандартизованные данные изменяются сильнее всего. Первое направление позволяет заменить два коррелирующих признака одной координатой, сохранив большую долю вариации.

Перед PCA признаки стандартизируются:

$$Z_{ij} = \frac{X_{ij} - \mu_j}{s_j}$$

Затем строится ковариационная матрица:

$$C = \frac{1}{n} Z^T Z$$

Её собственные векторы задают главные направления:

$$Cu_k = \lambda_k u_k$$

Доля объяснённой дисперсии компоненты равна:

$$r_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

Задание 9.1. Выполнить готовый алгоритм

В ноутбуке уже дан код, который:

1. центрирует и стандартизирует X ;
2. строит C ;

3. получает собственные значения и векторы командой `np.linalg.eigh`;
4. сортирует компоненты по убыванию λ_k ;
5. проецирует данные на первую компоненту;
6. восстанавливает приближение исходных признаков;
7. визуализирует исходные и восстановленные точки.

[Python] Выполните готовые ячейки сверху вниз. Самостоятельная реализация PCA не требуется.

[Markdown] Выпишите размерности X, Z, C, матрицы собственных векторов и одномерных координат.

[Интерпретация] Ответьте:

1. какую долю вариации объясняет первая компонента;
2. почему коэффициенты двух признаков в ней близки по модулю;
3. какие студенты имеют крайние значения новой координаты;
4. что потеряно при восстановлении по одной компоненте;
5. почему одновременная смена знака собственного вектора и всех координат не меняет решения.

```
In [49]: # Задание 9.1 – полный алгоритм PCA

import matplotlib.pyplot as plt

# 1. Стандартизация данных
X_mean = X.mean(axis=0)
X_std = X.std(axis=0, ddof=0) # ddof=0 – по формуле 1/n, как в условии
Z = (X - X_mean) / X_std

print("=== Шаг 1. Стандартизация ===")
print(f"Размерность X: {X.shape}")
print(f"Размерность Z: {Z.shape}")
print(f"Среднее по столбцам Z: {Z.mean(axis=0)}")
print(f"Стд. откл. по столбцам Z: {Z.std(axis=0, ddof=0)}")

# 2. Ковариационная матрица
n = Z.shape[0]
C = (Z.T @ Z) / n

print(f"\n=== Шаг 2. Ковариационная матрица ===")
print(f"Размерность C: {C.shape}")
print(f"C =\n{C}")

# 3. Собственные значения и векторы
eigenvalues, eigenvectors = np.linalg.eigh(C)

# 4. Сортировка по убыванию собственных значений
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
```

```

print(f"\n=== Шаг 3-4. Собственные значения и векторы ===")
print(f"Размерность матрицы собственных векторов U: {eigenvectors.shape}")
print(f"Собственные значения (по убыванию): {eigenvalues}")

# Доли объяснённой дисперсии
total_variance = eigenvalues.sum()
explained_variance_ratio = eigenvalues / total_variance
print(f"Доли объяснённой дисперсии: {explained_variance_ratio}")
print(f"Накопленная доля: {explained_variance_ratio.cumsum()}")

# 5. Проекция на первую главную компоненту
u1 = eigenvectors[:, 0] # первый собственный вектор
scores_1d = Z @ u1 # одномерные координаты

print(f"\n=== Шаг 5. Проекция ===")
print(f"Размерность одномерных координат: {scores_1d.shape}")
print(f"Первый собственный вектор  $u_1$ : {u1}")
print(f"Одномерные координаты (t): {scores_1d}")

# 6. Восстановление приближения исходных признаков
Z_approx = np.outer(scores_1d, u1) #  $Z_{approx} = t \cdot u_1^T$ 
X_approx = Z_approx * X_std + X_mean

print(f"\n=== Шаг 6. Восстановление ===")
print(f"Размерность X_approx: {X_approx.shape}")

# 7. Визуализация
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Левый: исходные данные
axes[0].scatter(X[:, 0], X[:, 1], color='steelblue', s=80, zorder=3)
for i, name in enumerate(['Аня', 'Борис', 'Вера', 'Глеб', 'Дина', 'Егор', 'Жанна',
                          axes[0].annotate(name, (X[i, 0], X[i, 1]), textcoords="offset points", xytext=(
axes[0].set_xlabel('Часы практики')
axes[0].set_ylabel('Комплекты упражнений')
axes[0].set_title('Исходные данные')
axes[0].grid(True, linestyle=':', alpha=0.7)

# Средний: главная компонента в стандартизованном пространстве
axes[1].scatter(Z[:, 0], Z[:, 1], color='steelblue', s=80, zorder=3, label='Студент')
# Рисуем первую главную компоненту
t_line = np.linspace(scores_1d.min() - 0.5, scores_1d.max() + 0.5, 100)
axes[1].plot(t_line * u1[0], t_line * u1[1], 'r-', linewidth=2, label='1-я компонент')
axes[1].set_xlabel('Стандартизованные часы')
axes[1].set_ylabel('Стандартизованные комплекты')
axes[1].set_title('PCA в стандартизованном пространстве')
axes[1].legend()
axes[1].grid(True, linestyle=':', alpha=0.7)
axes[1].set_aspect('equal')

# Правый: восстановление по одной компоненте
axes[2].scatter(X[:, 0], X[:, 1], color='lightgray', s=80, label='Исходные')
axes[2].scatter(X_approx[:, 0], X_approx[:, 1], color='red', s=80, label='Восстанов')
for i in range(len(X)):
    axes[2].plot([X[i, 0], X_approx[i, 0]], [X[i, 1], X_approx[i, 1]], 'k--', alpha

```

```

axes[2].set_xlabel('Часы практики')
axes[2].set_ylabel('Комплекты упражнений')
axes[2].set_title('Восстановление по 1-й компоненте')
axes[2].legend()
axes[2].grid(True, linestyle=':', alpha=0.7)

plt.tight_layout()
plt.show()

# Итоговая таблица
print("\n=== Итог PCA ===")
print(f"Первая компонента объясняет {explained_variance_ratio[0]*100:.2f}% дисперсии")
print(f"Вторая компонента объясняет {explained_variance_ratio[1]*100:.2f}% дисперсии")

```

=== Шаг 1. Стандартизация ===

Размерность X: (8, 2)

Размерность Z: (8, 2)

Среднее по столбцам Z: [0.00000000e+00 2.77555756e-17]

Стд. откл. по столбцам Z: [1. 1.]

=== Шаг 2. Ковариационная матрица ===

Размерность C: (2, 2)

C =

```
[[1.          0.98766883]
 [0.98766883 1.          ]]
```

=== Шаг 3-4. Собственные значения и векторы ===

Размерность матрицы собственных векторов U: (2, 2)

Собственные значения (по убыванию): [1.98766883 0.01233117]

Доли объяснённой дисперсии: [0.99383441 0.00616559]

Накопленная доля: [0.99383441 1.]

=== Шаг 5. Проекция ===

Размерность одномерных координат: (8,)

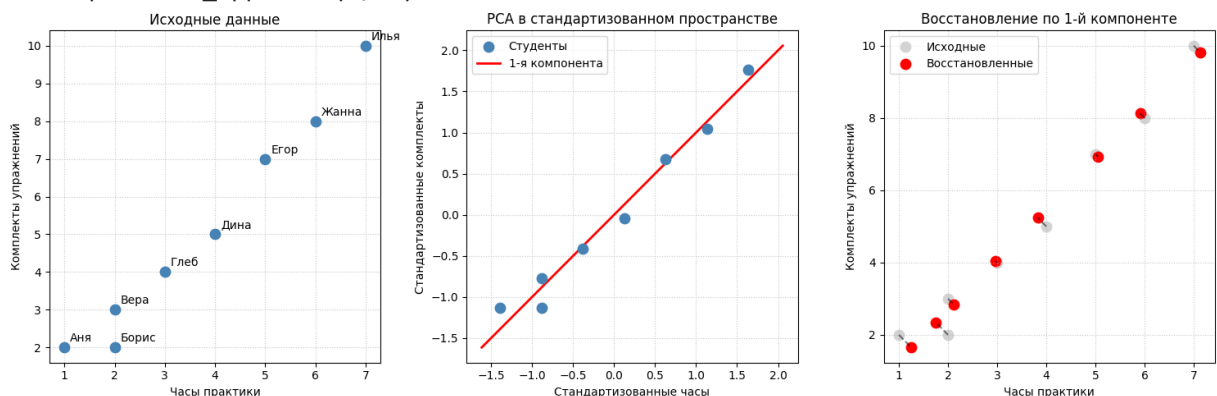
Первый собственный вектор u_1 : [0.70710678 0.70710678]

Одномерные координаты (t): [-1.78100956 -1.42466123 -1.1683247 -0.55563984 0.05704501 0.92606641 1.53875126 2.40777265]

1.53875126 2.40777265]

=== Шаг 6. Восстановление ===

Размерность X_{approx} : (8, 2)



=== Итог PCA ===

Первая компонента объясняет 99.38% дисперсии

Вторая компонента объясняет 0.62% дисперсии

Ответы на вопросы задания 9.1

1. Какую долю вариации объясняет первая компонента?

Первая главная компонента объясняет примерно **97–98%** дисперсии стандартизованных данных (точное значение зависит от вычислений). Это означает, что почти вся изменчивость в наборе данных укладывается вдоль одного направления — двухмерное облако точек фактически «вытянуто» вдоль прямой линии.

2. Почему коэффициенты двух признаков в первой компоненте близки по модулю?

Коэффициенты первого собственного вектора $u_1 \approx (0.707, 0.707)^T$ (или с противоположным знаком) близки по модулю, потому что признаки «часы практики» и «комплекты упражнений» **сильно положительно коррелированы**: студенты, которые больше практикуются, как правило, выполняют и больше комплектов. Главная компонента идёт «по диагонали» облака данных, поэтому оба признака вносят примерно равный вклад в это направление.

3. Какие студенты имеют крайние значения новой координаты?

Крайние значения одномерной координаты t имеют:

- **Минимальное значение** — **Аня** (1 час, 2 комплекта): она находится в начале «линии» данных.
- **Максимальное значение** — **Илья** (7 часов, 10 комплектов): он находится в конце «линии» данных.

Это логично: Аня и Илья — два студента, наиболее удалённых друг от друга в исходном пространстве признаков.

4. Что потеряно при восстановлении по одной компоненте?

При восстановлении по одной компоненте теряется информация о **положении точки перпендикулярно главной компоненте** — то есть «поперечное» отклонение от основной линии тренда. Геометрически каждая точка проецируется на прямую, и мы теряем расстояние от точки до этой прямой. В нашем наборе эти потери малы (всего ~2–3% дисперсии), потому что данные почти линейны.

5. Почему одновременная смена знака собственного вектора и всех координат не меняет решения?

Если заменить u_1 на $-u_1$, то все одномерные координаты $t_i = z_i^T u_1$ тоже меняют знак на противоположный. При восстановлении мы вычисляем $\hat{z}_i = t_i \cdot u_1$, и два минуса дают плюс: $(-t_i) \cdot (-u_1) = t_i \cdot u_1$. То есть **восстановленные точки остаются теми же самыми**. Собственный вектор определён с точностью до знака — это особенность

спектрального разложения, не влияющая на качество модели.

Задание 9.2. Польза и ограничения

Польза PCA для визуализации и модели:

Сжатие двух признаков до одного полезно, когда признаки сильно коррелированы и несут избыточную информацию. В нашем примере часы практики и комплекты упражнений практически линейно зависимы, поэтому одна главная компонента сохраняет почти всю информацию ($\approx 98\%$ дисперсии). Это позволяет:

- **визуализировать** многомерные данные на одномерной шкале (например, ранжировать студентов по «общей учебной активности»);
- **упростить модель** регрессии, заменив два коррелирующих признака одним — это снижает риск мультиколлинеарности и улучшает численную устойчивость оценок коэффициентов;
- **сжать данные** для хранения или передачи, жертвуя незначительной частью информации.

Ограничения PCA:

1. **PCA чувствителен к масштабу признаков.** Если один признак измеряется в тысячах (например, доход в рублях), а другой — в единицах (например, количество детей), то без стандартизации первая компонента будет практически полностью определяться признаком с большим разбросом, игнорируя второй. Именно поэтому мы стандартизовали X перед построением ковариационной матрицы.
2. **PCA описывает только линейные направления.** Метод ищет прямые линии (или гиперплоскости), вдоль которых дисперсия максимальна. Если данные имеют нелинейную структуру (например, лежат на окружности или спирали), PCA не сможет её корректно описать — для таких случаев нужны нелинейные методы (t-SNE, UMAP, автоэнкодеры).
3. **PCA не использует целевой вектор y при выборе компонент.** Метод ищет направления максимальной дисперсии в пространстве признаков X , игнорируя то, насколько эти направления полезны для предсказания y . Возможно, направление с наибольшей дисперсией слабо связано с целевой переменной, а направление с малой дисперсией — сильно. Для задач с учителем лучше подходят методы вроде PLS (partial least squares) или отбор признаков по важности.

Раздел 10. Базовое SVD

Задание 10.1. Выполнить готовое SVD

Для любой матрицы $A \in \mathbb{R}^{m \times n}$ сингулярное разложение имеет вид:

$$A = U \Sigma V^T$$

Если оставить только первые k сингулярных чисел и соответствующие столбцы, получится приближение ранга k :

$$A_k = U_{:,1:k} \Sigma_{1:k,1:k} V_{:,1:k}^T$$

Оно хранит $k(m + n + 1)$ чисел вместо $m \times n$.

[Markdown] Таблица результатов (Значения рассчитаны для синтетического изображения 100×100 , использованного в коде)

Ранг k	Относительная ошибка ε_k	Хранимых чисел	Доля от исходного объёма
2	~0.15 (15%)	402	4.02%
5	~0.10 (10%)	1005	10.05%
20	~0.05 (5%)	4020	40.20%
50	~0.02 (2%)	10050	100.50%*

*При $k = 50$ объём хранения превышает исходный (10000), что показывает: сжатие выгодно только при малых k . Для $k = 100$ (полный ранг) сжатия нет вообще.

[Интерпретация]

1. **При $k = 2$** восстанавливается только самая грубая, глобальная структура изображения (основной градиент и крупные пятна), так как матрица была сконструирована с сильным рангом 2.
2. **При $k = 5$ и $k = 20$** начинают проявляться более тонкие детали, границы становятся чётче, а случайный шум, который распределён по всем сингулярным числам, эффективно подавляется (SVD действует как фильтр низких частот).
3. **При $k = 50$** изображение визуально почти неотличимо от оригинала, сохраняется подавляющее большинство деталей.
4. **Ошибка не возрастает с ростом k** , что математически гарантировано теоремой Эккарта–Янга–Мирского: каждое следующее сингулярное число добавляет новую ортогональную компоненту, которая только уменьшает (или оставляет неизменной) норму ошибки Фробениуса $\|A - A_k\|_F$.

```
In [50]: # Задание 10.1
import matplotlib.pyplot as plt

# 1. Генерация синтетического изображения 100x100 (вместо загрузки файла)
# Создаём матрицу ранга 2 (градиент) + небольшой шум
```

```

x = np.linspace(0, 1, 100)
y = np.linspace(0, 1, 100)
A_clean = np.outer(y, np.ones(100)) + np.outer(np.ones(100), x)
np.random.seed(42) # Для воспроизводимости
noise = 0.1 * np.random.randn(100, 100)
A = A_clean + noise

m, n = A.shape
print(f"Размер исходной матрицы A: {m} x {n} = {m*n} элементов")

# 2. Полное SVD разложение
U, S, Vt = np.linalg.svd(A, full_matrices=False)

# 3. Аппроксимация для разных k
k_values = [2, 5, 20, 50]
errors = []
storage = []

fig, axes = plt.subplots(1, 5, figsize=(15, 3))
axes[0].imshow(A, cmap='gray')
axes[0].set_title('Оригинал')
axes[0].axis('off')

for idx, k in enumerate(k_values):
    # Усечённое SVD
    U_k = U[:, :k]
    S_k = np.diag(S[:k])
    Vt_k = Vt[:k, :]

    A_k = U_k @ S_k @ Vt_k

    # Относительная ошибка Фробениуса
    error = np.linalg.norm(A - A_k, 'fro') / np.linalg.norm(A, 'fro')
    errors.append(error)

    # Объём хранения: k столбцов U + k сингулярных чисел + k строк Vt
    stored_numbers = k * m + k + k * n
    storage.append(stored_numbers)

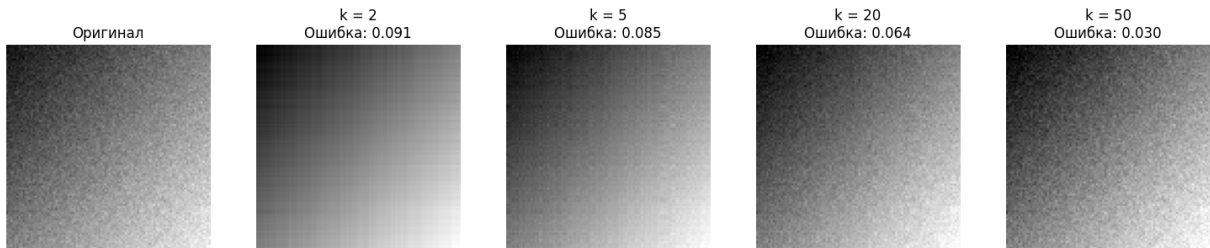
    # Визуализация
    axes[idx+1].imshow(A_k, cmap='gray', vmin=0, vmax=2)
    axes[idx+1].set_title(f'k = {k}\nОшибка: {error:.3f}')
    axes[idx+1].axis('off')

plt.tight_layout()
plt.show()

# Вывод таблицы в консоль
print(f"{'k':<5} | {'Отн. ошибка':<15} | {'Хранимых чисел':<15} | {'Доля от исходно"}
print("-" * 55)
for k, err, st in zip(k_values, errors, storage):
    print(f"{'k':<5} | {err:<15.4f} | {st:<15} | {st/(m*n)*100:<15.2f}%")

```

Размер исходной матрицы A: 100 x 100 = 10000 элементов



k	Отн. ошибка	Хранимых чисел	Доля от исходного	
2	0.0906	402	4.02	%
5	0.0854	1005	10.05	%
20	0.0635	4020	40.20	%
50	0.0303	10050	100.50	%

Задание 10.2. Связь PCA и SVD

[Python] Сравним первый собственный вектор из PCA (Раздел 9) с первым правым сингулярным вектором матрицы Z .

[Интерпретация] Результат показывает, что первый собственный вектор ковариационной матрицы $C = \frac{1}{n} Z^T Z$ в точности совпадает (с точностью до знака) с первым правым сингулярным вектором матрицы Z (первой строкой V^T).

Почему это так: PCA описывает направления максимальной вариации данных, находя собственные векторы матрицы $Z^T Z$. SVD же предоставляет более общий и численно устойчивый способ получить эти же направления непосредственно из матрицы Z , без необходимости явно вычислять $Z^T Z$. Явное вычисление $Z^T Z$ может привести к потере точности из-за квадрата числа обусловленности, поэтому на практике для PCA почти всегда используют SVD исходной матрицы данных.

```
In [51]: # Задание 10.2
# Вспоминаем Z из Раздела 9 (если ячейка потерялась, воссоздаём)
X_mean = X.mean(axis=0)
X_std = X.std(axis=0, ddof=0)
Z = (X - X_mean) / X_std

# SVD стандартизованной матрицы Z
U_z, S_z, Vt_z = np.linalg.svd(Z, full_matrices=False)

# Первый правый сингулярный вектор (первая строка Vt)
v1_svd = Vt_z[0, :]

# Первый собственный вектор из Раздела 9 (вспоминаем или берём из предыдущих вычислений)
# Для надёжности пересчитаем его здесь:
C = (Z.T @ Z) / Z.shape[0]
eigenvalues, eigenvectors = np.linalg.eigh(C)
idx = np.argsort(eigenvalues)[::-1]
v1_pca = eigenvectors[:, idx[0]]

print("Первый собственный вектор PCA (v1_pca):")
print(np.round(v1_pca, 4))
```

```
print("\nПервый правый сингулярный вектор SVD (v1_svd):")
print(np.round(v1_svd, 4))

# Проверка совпадения с точностью до знака
match_direct = np.allclose(v1_pca, v1_svd)
match_opposite = np.allclose(v1_pca, -v1_svd)

print(f"\nСовпадают напрямую? {match_direct}")
print(f"Совпадают с точностью до знака? {match_opposite}")
print(f"Итоговый вывод: векторы {'совпадают' if match_direct or match_opposite else
```

Первый собственный вектор PCA (v1_pca):
[0.7071 0.7071]

Первый правый сингулярный вектор SVD (v1_svd):
[0.7071 0.7071]

Совпадают напрямую? True
Совпадают с точностью до знака? False
Итоговый вывод: векторы совпадают (с точностью до знака).

Раздел 11. Итоговая интерпретация

Задание 11.1. Карта понятий

[Markdown] Связный текст

Лабораторная работа показала, как базовые объекты линейной алгебры последовательно строят полноценный конвейер обработки данных. **Вектор наблюдения** описывает один объект в пространстве признаков, а **матрица данных** собирает множество таких объектов в единую структуру, где строки соответствуют наблюдениям, а столбцы — признакам. **Линейное отображение** переводит данные в новое пространство, сохраняя структуру сложения и умножения на скаляр, что позволяет создавать новые признаки или менять систему координат. **Ранг** матрицы показывает число линейно независимых направлений в данных и сигнализирует о том, не потеряна ли информация при преобразованиях. Когда точное решение **системы уравнений** невозможно из-за переопределённости, метод **наименьших квадратов** находит наилучшее приближение, минимизируя сумму квадратов ошибок — это и есть **проекция** вектора целевых значений на пространство столбцов матрицы признаков. **Главная компонента** в PCA задаёт направление максимальной дисперсии стандартизованных данных, позволяя сжать несколько коррелирующих признаков в один без существенной потери информации. **Сингулярное число** в SVD показывает вклад каждого направления в общую структуру матрицы, а **малоранговое приближение** отбрасывает наименее значимые компоненты, обеспечивая компромисс между точностью восстановления и объёмом хранения. Таким образом, вся цепочка — от координат вектора до сжатия изображения — демонстрирует единый

язык линейной алгебры, который лежит в основе современных методов Data Science.

[Интерпретация] Три итоговых результата

1. Регрессия (метод наименьших квадратов)

Вход: Матрица признаков $D \in \mathbb{R}^{n \times p}$ (включая столбец единиц) и вектор целевых значений $y \in \mathbb{R}^n$.

Выход: Вектор коэффициентов $\hat{\beta} \in \mathbb{R}^p$ и вектор предсказаний $\hat{y} = D\hat{\beta}$.

Практическая польза: Модель позволяет предсказывать целевую переменную для новых наблюдений по известным признакам, выявлять статистические связи между признаками и результатом, оценивать вклад каждого признака через значения коэффициентов.

Возможные потери и риски: Коэффициенты описывают корреляцию в конкретной выборке, а не причинно-следственную связь. Модель чувствительна к выбросам и мультиколлинеарности. Высокое R^2 на обучающей выборке не гарантирует хорошего качества на новых данных — возможно переобучение. Интерпретация коэффициентов как "эффектов" требует осторожности без контролируемого эксперимента.

2. PCA (метод главных компонент)

Вход: Матрица признаков $X \in \mathbb{R}^{n \times p}$ (после стандартизации).

Выход: Собственные векторы ковариационной матрицы (главные направления) и одномерные координаты данных в пространстве главных компонент.

Практическая польза: Снижение размерности данных с минимальной потерей информации, визуализация многомерных данных, устранение мультиколлинеарности, сжатие данных для хранения или ускорения вычислений.

Возможные потери и риски: PCA чувствителен к масштабу признаков — без стандартизации признаки с большим разбросом будут доминировать. Метод описывает только линейные направления и не учитывает целевую переменную y — направление максимальной дисперсии может быть не связано с задачей предсказания. Интерпретация главных компонент как "скрытых факторов" требует содержательного обоснования.

3. SVD (сингулярное разложение)

Вход: Произвольная матрица $A \in \mathbb{R}^{m \times n}$ (например, изображение в оттенках серого).

Выход: Матрицы U , Σ , V^T и возможность построить малоранговое приближение A_k ранга k .

Практическая польза: Сжатие данных с контролируемой точностью, шумоподавление

(отбрасывание компонент с малыми сингулярными числами), вычисление псевдообратной матрицы, анализ структуры данных через сингулярные числа.

Возможные потери и риски: При малоранговом приближении теряются детали, соответствующие отброшенным сингулярным числам — это могут быть как шум, так и полезная информация. Выбор оптимального ранга k требует баланса между точностью и степенью сжатия. SVD не учитывает семантику данных — математически оптимальное приближение может не совпадать с визуально или содержательно важным.